AVCC Electronics System Architecture

Detailed Technical Reference

Platform:	ESP32 (Espressif, Arduino Core 2.0.14 / ESP-IDF 5.x)
Protocol:	ESP-NOW (250 kbps, 250 byte max payload)
Topology:	1 Master + 3 Slaves (star)
Channels:	12 per slave (36 total outputs)

Generated March 8, 2026

Contents

1 System Overview **2**
- 1.1 High-Level Topology . . . 2
- 1.2 Communication Flow Summary . . . 2

2 Wire Protocol **3**
- 2.1 Command Message (`Msg`) — Master → Slaves . . . 3
  - 2.1.1 Command Codes . . . 3
  - 2.1.2 Flag Bits . . . 3
- 2.2 Acknowledgment Message (`AckMsg`) — Slaves → Master . . . 4
- 2.3 Status Message (`StatusMsg`) — Slaves → Master . . . 4
- 2.4 Sequence Guard Mechanism . . . 4

3 Master Controller **4**
- 3.1 Wi-Fi and Web Server . . . 4
  - 3.1.1 HTTP Endpoints . . . 5
- 3.2 NVS Persistent Storage . . . 5
- 3.3 Master State Machine . . . 5
- 3.4 Burst Send Architecture . . . 6
- 3.5 Targeted Retransmission . . . 7
- 3.6 Keepalive . . . 7
- 3.7 Per-Channel Settings . . . 7

4 Electrical Slave **7**
- 4.1 Dual-Core Task Architecture . . . 8
  - 4.1.1 Adaptive Yield Strategy . . . 8
- 4.2 DAC Hardware Interface . . . 8
  - 4.2.1 Pin Mapping . . . 8
  - 4.2.2 Write Sequence . . . 9
- 4.3 Per-Channel State Machine . . . 9
- 4.4 Inter-Task Communication . . . 10
- 4.5 Timing Constants . . . 10

5 Pressure Slave **10**
- 5.1 GPIO Pin Mapping . . . 10
- 5.2 Per-Channel State Machine . . . 10
- 5.3 Duration Auto-Stop . . . 11
- 5.4 FreeRTOS Message Queue . . . 11
- 5.5 Architecture Differences from Electrical . . . 11

6 Servo Slave **11**
- 6.1 GPIO Pin Mapping . . . 11
- 6.2 Motion Profile . . . 12
- 6.3 Per-Channel State Machine . . . 12
- 6.4 2 Hz Hard Cap . . . 12
- 6.5 Cycle Boundary Logic . . . 12

7 Synchronization and Timing **13**
- 7.1 Start Delay and Phase Offset . . . 13
- 7.2 Keepalive Failsafe . . . 13
- 7.3 Peer Management . . . 13

8 Safety Mechanisms **14**
8.1 Failsafe Summary 14
8.2 Safe Stop Behavior 14

9 Memory and Performance **14**
9.1 Web Server Heap Management 14
9.2 Queue Sizing 14

10 Utility Sketches **15**
10.1 BICEP_MAC_Address 15
10.2 Test_on_Wifi 15

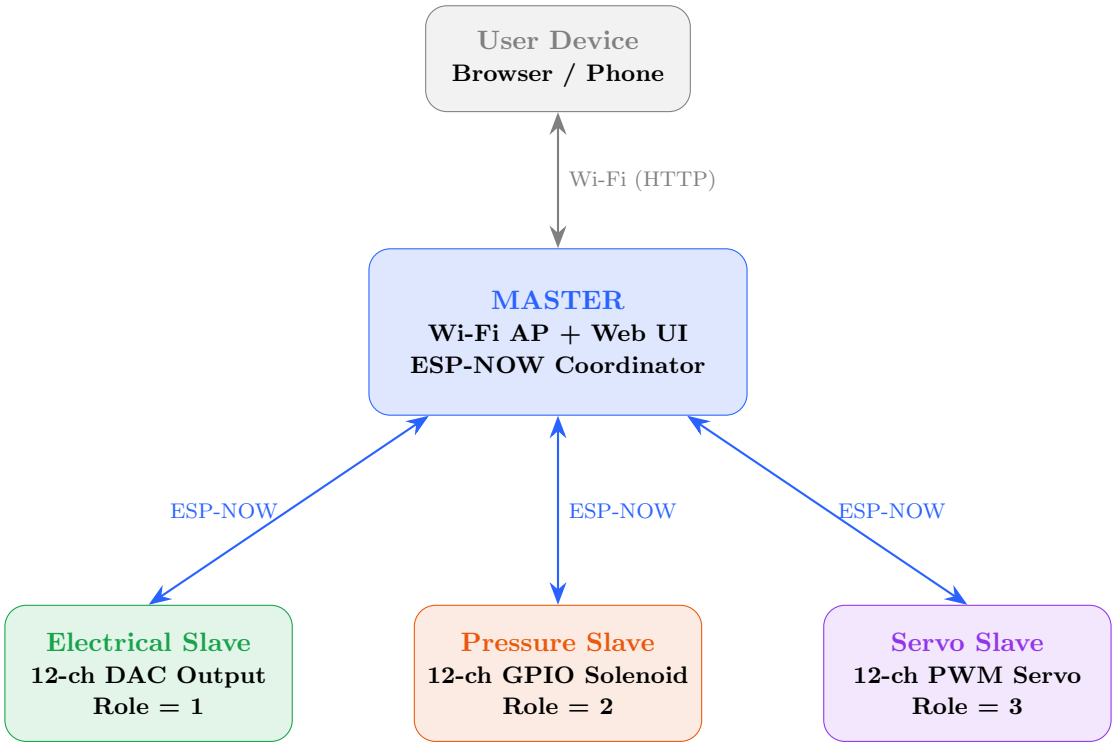
11 Complete File Reference **15**

12 Appendix: MAC Addresses (Current Configuration) **15**

1 System Overview

The AVCC (Autonomous Vascular Compression Controller) Electronics system is a real-time, wireless, multi-channel waveform controller built on four ESP32 microcontrollers connected via ESP-NOW. It drives three different output modalities—analog voltage (electrical), digital solenoid (pressure), and servo motor—synchronized from a single master controller.

1.1 High-Level Topology



1.2 Communication Flow Summary

1. **User** connects to the Master's Wi-Fi AP (`AVCC_Master 5`, password `12345678`) and opens `http://192.168.4.1`.
2. **Master** serves a streamed HTML control page where the user configures 12 channels (frequency, pulse width, amplitude, pressure duration, phase offsets).
3. On **Apply + Start**, the Master enters the `MS_WAIT_ACKS` state, burst-sends `CMD_SET` messages for all 12 channels to all 3 slaves, followed by targeted `CMD_START` messages.
4. Each **Slave** receives messages via a FreeRTOS queue, applies per-channel configuration, and sends back `AckMsg` (0xA1) confirmations.
5. During operation (`MS_RUNNING`), the Master sends `CMD_KEEPALIVE` at 6.7 Hz. Slaves auto-stop if no keepalive arrives within 500 ms.
6. Slaves send `StatusMsg` (0xA2) at 1 Hz with health telemetry (heap, heartbeat age, sequence watermark).

2 Wire Protocol

All messages are sent as raw ESP-NOW payloads. Structs use `__attribute__((packed))` to eliminate padding and guarantee byte-exact layout across devices.

2.1 Command Message (`Msg`) — Master → Slaves

Field	Type	Offset	Description
`cmd`	`uint8_t`	0	Command code (1–4)
`ch`	`uint8_t`	1	Channel 1–12 (0 = all)
`flags`	`uint8_t`	2	Bitfield (see below)
`neutral_code`	`uint8_t`	3	DAC neutral code (0x80)
`run_id`	`uint32_t`	4	Monotonic run identifier
`seq`	`uint32_t`	8	Per-channel sequence number
`period_us`	`uint32_t`	12	Full cycle period (μs)
`pulse_us`	`uint32_t`	16	Pulse width (μs)
`pos_code`	`uint8_t`	20	Positive DAC code
`neg_code`	`uint8_t`	21	Negative DAC code
`rsv0`	`uint8_t`	22	Reserved
`rsv1`	`uint8_t`	23	Reserved
`start_delay_us`	`uint32_t`	24	Sync start delay (μs)
`phase_offset_us`	`uint32_t`	28	Phase offset (μs)
`duration_ms`	`uint32_t`	32	Run duration (ms); 0 = forever

Total size: 36 bytes (V3). The legacy V1 format is 24 bytes (no `start_delay_us`, `phase_offset_us`, `duration_ms`). Pressure and Servo slaves auto-detect V1 by checking `len == sizeof(MsgV1)`.

2.1.1 Command Codes

Value	Name	Description
1	`CMD_SET`	Configure channel parameters (period, pulse, amplitude). Does not start output.
2	`CMD_START`	Begin waveform output. Carries start delay, phase offset, and duration.
3	`CMD_STOP`	Immediately stop all channels and park outputs at safe state.
4	`CMD_KEEPALIVE`	Heartbeat ping. Resets each slave's failsafe timer.

2.1.2 Flag Bits

Bit	Constant	Meaning
0	`FLG_SET_MIRROR` (0x01)	Mirror waveform around neutral (electrical only)
1	`FLG_TGT_ELEC` (0x02)	START targets the Electrical slave
2	`FLG_TGT_PRES` (0x04)	START targets the Pressure slave
3	`FLG_TGT_MOTO` (0x08)	START targets the Servo slave

2.2 Acknowledgment Message (AckMsg) — Slaves → Master

Field	Type	Offset	Description
type	uint8_t	0	Always 0xA1
role	uint8_t	1	1=Electrical, 2=Pressure, 3=Motor
cmd	uint8_t	2	Echoed command code
ch	uint8_t	3	Echoed channel (0 for STOP)
run_id	uint32_t	4	Echoed run ID
seq	uint32_t	8	Echoed sequence number
ok	uint8_t	12	1 = accepted, 0 = rejected
rsv[3]	uint8_t	13	Reserved padding

Total size: 16 bytes.

2.3 Status Message (StatusMsg) — Slaves → Master

Field	Type	Offset	Description
type	uint8_t	0	Always 0xA2
role	uint8_t	1	Slave role identifier
rsv0, rsv1	uint8_t	2	Reserved
millis_now	uint32_t	4	Slave's millis() timestamp
run_id	uint32_t	8	Current run ID on slave
last_seq_max	uint32_t	12	Highest sequence number seen
hb_age_ms	uint16_t	16	Keepalive age (ms), capped at 65535
free_heap	uint16_t	18	Free heap (bytes), capped at 65535

Total size: 20 bytes. Sent at 1 Hz by each slave.

2.4 Sequence Guard Mechanism

Each slave maintains a per-channel sequence counter `lastSeq[12]`. When a `CMD_SET` or `CMD_START` arrives, the slave checks:

```
static inline bool acceptSeq(uint8_t idx0, const Msg &m) {
  if (m.seq <= lastSeq[idx0]) return false; // stale -> reject
  lastSeq[idx0] = m.seq;
  if (m.seq > lastSeqMax) lastSeqMax = m.seq;
  return true;
}
```

When a new `run_id` is detected (master restarted), all sequence counters are reset to 0 so fresh commands are accepted.

3 Master Controller

File: `Master_Pressure_Servo_Electrical.ino` (1049 lines)

3.1 Wi-Fi and Web Server

The Master runs in `WIFI_AP` mode (not `AP_STA`) to avoid radio instability on ESP-IDF 5.x. It creates a soft access point:

Parameter	Value
SSID	`AVCC_Master 5`
Password	`12345678`
Channel	1
Max clients	4
IP address	`192.168.4.1`

The web server uses **streamed HTML** (chunked transfer encoding) via `server.sendContent()` to avoid heap-killing `String` concatenation. All HTML is built with stack-allocated `char buf[]` arrays and `snprintf()`.

`WEBSERVER_MAX_POST_ARGS` is set to **100** (default 32) because the form has $12 \times 7 = 84$ channel fields plus control buttons.

3.1.1 HTTP Endpoints

Method	Path	Handler
GET	`/`	`handleRoot()` – serve control page
POST	`/apply`	`handleApply()` – apply/start/stop
POST	`/save`	`handleSave()` – save to NVS flash
POST	`/load`	`handleLoad()` – reload from NVS
POST	`/factory`	`handleFactory()` – factory defaults (RAM)
POST	`/factory_save`	`handleFactorySave()` – factory + save
GET	`/favicon.ico`	`handleFavicon()` – 204 No Content

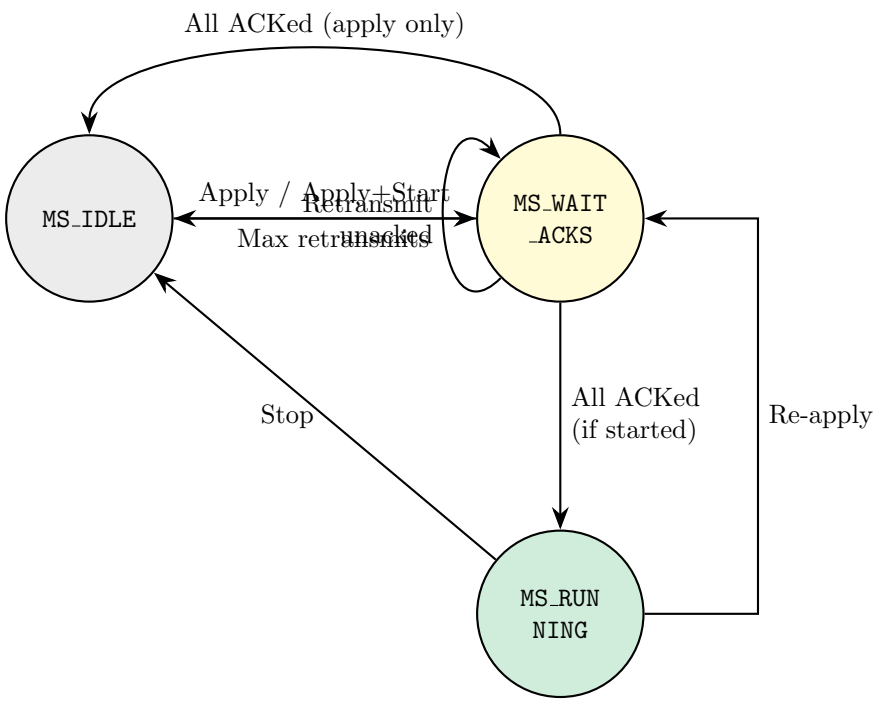
3.2 NVS Persistent Storage

Settings are stored in the ESP32's Non-Volatile Storage (NVS) partition under namespace `"avcc"`. Per-channel keys use a compact naming scheme:

Key pattern	Type	Content
`en%02d`	Bool	Channel enable
`f%02d`	Float	Frequency (Hz)
`pu%02d`	UInt	Pulse width (μs)
`a%02d`	Float	Amplitude (V)
`pd%02d`	UInt	Pressure duration (ms)
`pp%02d`	UInt	Pressure phase offset (μs)
`mp%02d`	UInt	Motor phase offset (μs)
`dbg`	Bool	Debug serial output
`boot`	Bool	Boot auto-start

3.3 Master State Machine

The Master uses an explicit three-state machine to manage the apply/start/stop lifecycle:



Listing 1: Master state types

```
enum MasterState : uint8_t {
  MS_IDLE,      // waiting for user input
  MS_WAIT_ACKS, // sent SETs (and maybe STARTs), waiting for ACKs
  MS_RUNNING    // all channels active, sending keepalives
};

enum RequestType : uint8_t {
  REQ_APPLY_ONLY, // send SETs only, return to IDLE
  REQ_APPLY_START, // send SETs + STARTs, transition to RUNNING
  REQ_STOP         // send STOP, return to IDLE
};

struct PendingRequest {
  RequestType type;
  bool valid = false;
};
```

3.4 Burst Send Architecture

Instead of sending one channel per `loop()` iteration (the old `applyOneStep()` pattern), the Master now sends all 12 channels in a single burst:

1. `burstAllSets()` — Iterates over all 12 channels. For each enabled channel, increments its sequence number, constructs a `CMD_SET` message, sends it to all 3 slaves via `sendSetToAll()`, and sets the corresponding bit in `unackedBitmap[3]`. Calls `yield()` after each channel to let the ESP-NOW TX queue drain.

2. `burstAllStarts()` — For each enabled channel, sends 3 targeted `CMD_START` messages (one per slave) with appropriate `flags`, `phase_offset_us`, and `duration_ms`. Each message gets a fresh sequence number.

3.5 Targeted Retransmission

The Master tracks acknowledgments per-slave, per-channel:

```
struct SlaveHealth {
  uint32_t ackSeqCh[12]; // last acked seq per channel
  uint32_t expSeqCh[12]; // expected seq per channel
  // ...health telemetry...
};
SlaveHealth H[3]; // S_ELEC=0, S_PRES=1, S_MOTO=2
uint16_t unackedBitmap[3]; // per-slave bitmask of unacked channels
```

`retransmitUnacked()` only re-sends `CMD_SET` messages to slaves that haven't acknowledged specific channels, rather than broadcasting to all. Up to `MAX_RETRANSMITS` = 3 retries are attempted at 600 ms intervals.

3.6 Keepalive

When `running == true`, the Master sends `CMD_KEEPALIVE` to all 3 slaves every 150 ms (≈6.7 Hz). This rate is deliberately lower than the original 20 Hz to reduce congestion with 3 slaves.

3.7 Per-Channel Settings

Each of the 12 channels has:

Parameter	Range	Default
Frequency (Hz)	0.01–10	0.5
Pulse width (μs)	50–$T/2$	2000
Amplitude (V)	0–15	15.0
Pressure duration (ms)	0+	330
Pressure phase offset (μs)	0+	0
Motor phase offset (μs)	0+	0
Enable	bool	true

The amplitude is converted to an 8-bit DAC code using:

$$\texttt{code} = \text{round}\left(\frac{V + 15}{30} \times 255\right)$$

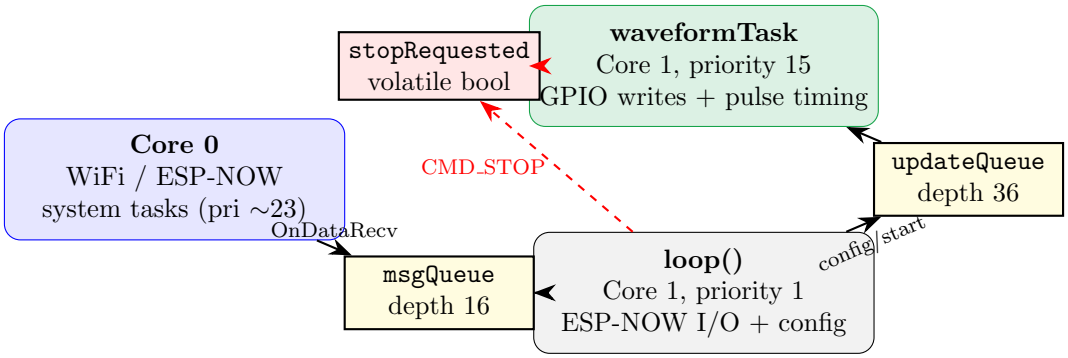
where $V \in [-15, +15]$ volts, giving code 0x00 for -15 V and 0xFF for $+15$ V, with 0x80 as the neutral midpoint.

4 Electrical Slave

File: `Electrical_Receiver_V2.ino` (755 lines)

This slave generates bipolar analog waveforms on 12 channels via an 8-bit parallel DAC bus. It is the most architecturally complex slave due to its dual-task design.

4.1 Dual-Core Task Architecture



- **Core 0** runs WiFi and ESP-NOW system tasks at priority ∼23. The `OnDataRecv` callback executes here but *only* enqueues packets into `msgQueue`—no processing.
- **Core 1** hosts two tasks:
  - `waveformTask` (priority 15): owns all GPIO writes, the pulse scheduler, and the `ChanState[12]` array. Spin-waits when the next edge is $< 500\,\mu$s away for sub-millisecond timing precision.
  - `loop()` (priority 1): handles all ESP-NOW I/O (receive, ACK, status), config parsing, and cross-task communication via `updateQueue`.

4.1.1 Adaptive Yield Strategy

`waveformTask` dynamically adjusts its yield behavior based on the time until the next scheduled edge:

Gap to next edge	Action	Rationale
$\leq 500\,\mu$s	Spin-wait (no yield)	Sub-ms critical window
$500\,\mu$s – 2 ms	`taskYIELD()`	Brief reschedule
2 ms – 5 ms	`vTaskDelay(1)`	One FreeRTOS tick
> 5 ms	`vTaskDelay(gap−2ms)`	Long sleep
No active channels	`vTaskDelay(10)`	Deep idle

4.2 DAC Hardware Interface

The Electrical slave drives 12 DAC channels through 3 quad-DAC chips connected via an 8-bit parallel data bus:

4.2.1 Pin Mapping

Signal	GPIO(s)	Purpose
DB0–DB7	16, 15, 14, 13, 12, 5, 4, 2	8-bit data bus
DAC_SEL0	17	DAC channel select bit 0
DAC_SEL1	18	DAC channel select bit 1
WR1	23	Write strobe, chip 1 (ch 1–4)
WR2	25	Write strobe, chip 2 (ch 5–8)
WR3	26	Write strobe, chip 3 (ch 9–12)

4.2.2 Write Sequence

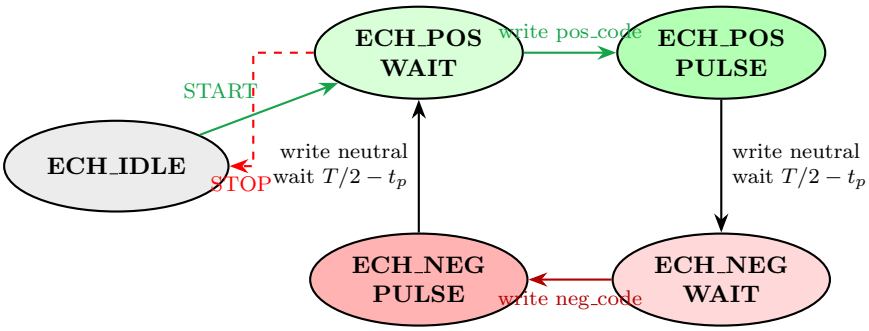
To set a channel's DAC output:

1. Apply logical-to-physical channel remapping (swaps channels 2↔3, 6↔7, 10↔11).
2. Write the 8-bit code to the data bus using direct GPIO register writes (`GPIO.out_w1tc` / `GPIO.out_w1ts`).
3. Wait 1 μs (data setup time).
4. Set the DAC select lines (2-bit address within chip).
5. Wait 1 μs (select setup time).
6. Strobe the appropriate WR pin LOW for 2 μs, then HIGH (latch).

Total write time: ≈4–5 μs per channel update.

4.3 Per-Channel State Machine

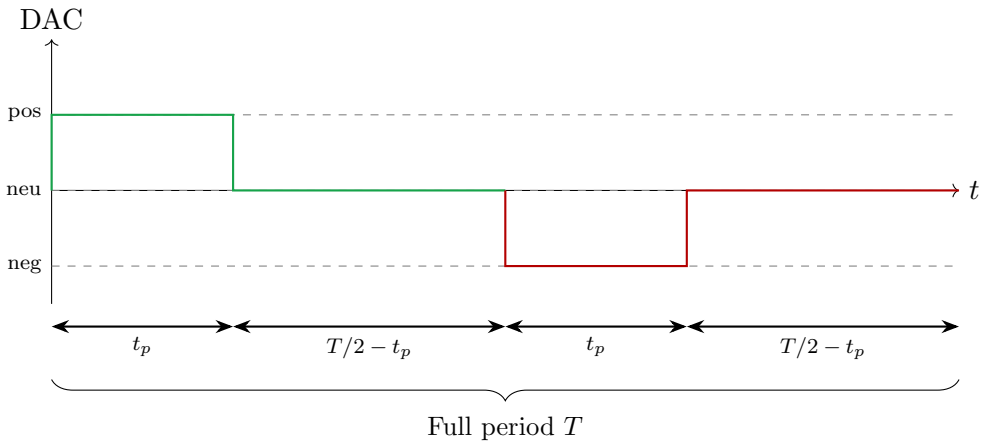
Each electrical channel follows a 5-state machine for bipolar waveform generation:



Listing 2: Electrical channel state enum

```
enum ElecChState : uint8_t {
  ECH_IDLE,      // channel disabled
  ECH_POS_WAIT,  // waiting for positive pulse edge
  ECH_POS_PULSE, // positive pulse active (DAC = pos_code)
  ECH_NEG_WAIT,  // waiting for negative pulse edge
  ECH_NEG_PULSE  // negative pulse active (DAC = neg_code)
};
```

Waveform timing (one full cycle):



Where t_p = `pulse_us` and T = `period_us`.

4.4 Inter-Task Communication

Queue	Direction	Depth	Payload
`msgQueue`	ESP-NOW CB → `loop()`	16	`RxPacket`: 6-byte MAC + 36-byte `Msg`
`updateQueue`	`loop()` → `waveformTask`	36	`ChanUpdate`: type + channel config + nextEdge
`stopRequested`	`loop()` → `waveformTask`	1 (flag)	`volatile bool`

4.5 Timing Constants

Constant	Value	Purpose
`MIN_PULSE_US`	50 μs	Minimum pulse width
`MIN_PERIOD_US`	100 ms	Maximum frequency (10 Hz)
`MAX_PERIOD_US`	30 s	Minimum frequency (0.033 Hz)
`HEARTBEAT_TIMEOUT_MS`	500 ms	Failsafe timeout
`STATUS_PERIOD_MS`	1000 ms	Health report interval
`MAX_EVENTS_PER_BURST`	400	Scheduler burst limit

5 Pressure Slave

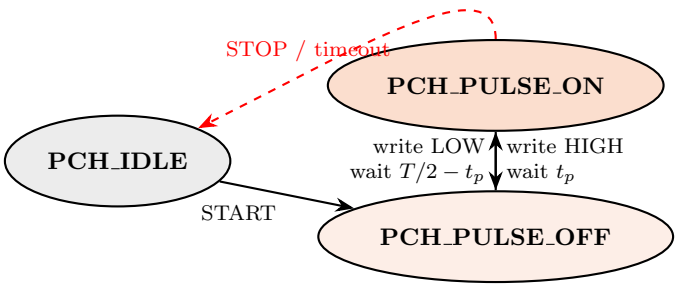
File: `Pressure_Receiver_V2.ino` (506 lines)

This slave controls 12 solenoid valves via digital GPIO outputs. It generates a square wave by toggling each output HIGH/LOW at a rate of **two pulses per full electrical period** (one pulse per half-cycle).

5.1 GPIO Pin Mapping

CH1	CH2	CH3	CH4	CH5	CH6	CH7	CH8	CH9	CH10	CH11	CH12
19	14	15	23	25	26	0	2	4	5	12	13

5.2 Per-Channel State Machine



Listing 3: Pressure channel state enum

```
enum PressChState : uint8_t {
  PCH_IDLE,      // disabled or not started
  PCH_PULSE_ON,  // output HIGH (pulse active)
  PCH_PULSE_OFF  // output LOW (off-portion of half-cycle)
```

```
};
```

5.3 Duration Auto-Stop

Each channel has a `stopAt_ms` field. When `duration_ms` is non-zero in the `CMD_START` message, the slave computes `stopAt_ms = millis() + duration_ms`. Each `loop()` iteration checks for expired channels:

```
if (chs[i].chState != PCH_IDLE &&
    chs[i].stopAt_ms != 0 &&
    (int32_t)(nowms - chs[i].stopAt_ms) >= 0) {
  chs[i].chState = PCH_IDLE;
  writePressure(i+1, false);
}
```

5.4 FreeRTOS Message Queue

The Pressure slave uses a FreeRTOS queue (depth 16) to safely pass messages from the ESP-NOW receive callback to the main loop, preventing packet drops during the Master's burst send of 12 channels:

```
struct RxPacket { uint8_t mac[6]; Msg msg; };
static QueueHandle_t msgQueue = nullptr;
static const int MSG_QUEUE_DEPTH = 16;
```

The callback (`OnDataRecv`) also supports backward-compatible V1 messages by detecting `len == sizeof(MsgV1)` and zero-filling the V3 extension fields.

5.5 Architecture Differences from Electrical

Aspect	Electrical	Pressure
Threading	Dual-task (waveformTask + loop)	Single-task (loop only)
Output	8-bit DAC (analog)	Digital GPIO (HIGH/LOW)
Pulses/cycle	2 (bipolar: pos + neg)	2 (both HIGH)
Stop method	`stopRequested` flag	Direct `safeStopAll()`
Duration	Not supported	Per-channel auto-stop

6 Servo Slave

File: `Servo_Receiver_V2.ino` (494 lines)

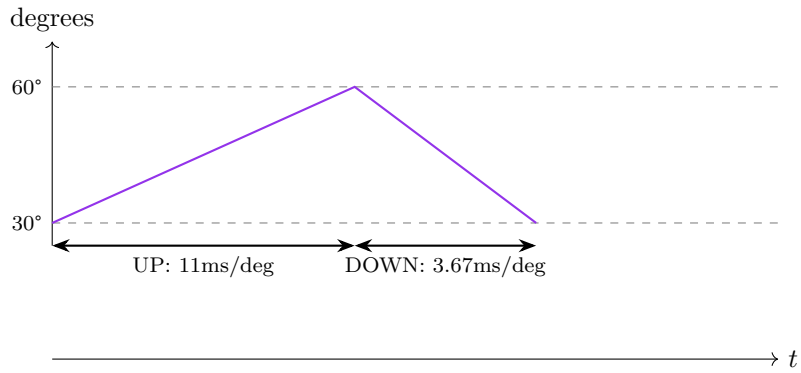
This slave drives 12 hobby servo motors with a ramp-up/ramp-down motion profile. It has a 2 Hz hard frequency cap to protect the mechanical servos.

6.1 GPIO Pin Mapping

Same physical pins as the Pressure slave:

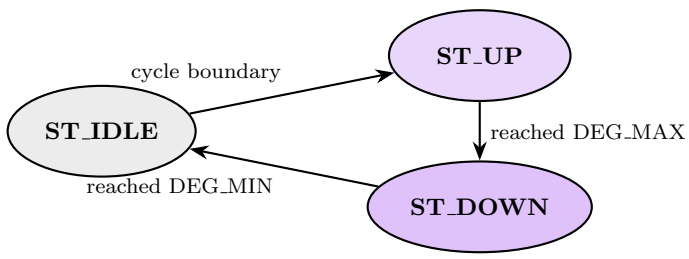
CH1	CH2	CH3	CH4	CH5	CH6	CH7	CH8	CH9	CH10	CH11	CH12
19	14	15	23	25	26	0	2	4	5	12	13

6.2 Motion Profile



Parameter	Value	Description
`DEG_MIN`	30°	Motion start/end angle
`DEG_MAX`	60°	Motion peak angle
`UP_STEP_US`	11 000 μs	Time per degree during ramp-up
`DOWN_STEP_US`	3 667 μs	Time per degree during ramp-down
Range	30°	Total sweep = 30 degrees
Up time	∼330 ms	30 × 11 ms
Down time	∼110 ms	30 × 3.67 ms

6.3 Per-Channel State Machine



The servo uses the existing `MotionStage` enum (unchanged from the original design):

```
enum MotionStage : uint8_t {
  ST_IDLE = 0,
  ST_UP   = 1,
  ST_DOWN = 2
};
```

6.4 2 Hz Hard Cap

If the configured frequency exceeds 2 Hz (`period_us < 500000`), the channel is disabled and the `CMD_START` ACK is sent with `ok = 0` (rejected). This protects the servo mechanism from being driven faster than it can physically respond.

6.5 Cycle Boundary Logic

Unlike the Electrical and Pressure slaves which schedule edge events via `nextEdge_us`, the Servo slave uses a two-level scheduler:

1. **Cycle boundary**: Each channel has a `nextCycle_us` timestamp. When the current time passes this boundary, a new motion cycle is triggered (if the channel is idle) by calling `startMotion()`.
2. **Step scheduler**: Within a motion cycle, each channel has a `nextStep_us` timestamp. The servo angle is incremented/decremented by 1° at each step event.

7 Synchronization and Timing

7.1 Start Delay and Phase Offset

When the Master sends `CMD_START`, each slave computes its first edge as:

$$\texttt{nextEdge_us} = \texttt{micros()} + \texttt{start_delay_us} + \texttt{phase_offset_us}$$

- `start_delay_us` is a global constant (600 ms default) that gives all slaves time to receive and process their START messages before any output begins.
- `phase_offset_us` is per-channel, per-slave, allowing the user to shift pressure and motor outputs relative to the electrical waveform.

7.2 Keepalive Failsafe

All 3 slaves implement the same heartbeat failsafe:

Parameter	Value
Master send rate	Every 150 ms (∼6.7 Hz)
Slave timeout	500 ms (∼3 missed keepalives)
Action on timeout	`safeStopAll()` — all outputs to safe state

Implementation difference: The Electrical slave cannot call `safeStopAll()` directly from `loop()` because that function writes to GPIOs owned by `waveformTask` on the same core at priority 15. Instead, it sets `stopRequested = true` and the waveform task checks this flag at the top of each iteration. Pressure and Servo slaves are single-threaded, so they call `safeStopAll()` directly.

7.3 Peer Management

Master (AP mode): Adds all 3 slave MACs as peers at startup using `WIFI_IF_AP` interface. Channel is set to 0 (use current).

Slaves (STA mode): Dynamically add the Master as a peer when they receive the first packet. The Master's MAC is extracted from the `esp_now_recv_info_t` in the receive callback. A send-failure backoff mechanism re-registers the peer after 3 consecutive `esp_now_send` failures:

```
static inline void trackSendResult(esp_err_t e) {
  if (e == ESP_OK) { sendFailCount = 0; }
  else { if (++sendFailCount >= 3) masterPeerAdded = false; }
}
```

8 Safety Mechanisms

8.1 Failsafe Summary

Mechanism	Description
Heartbeat timeout	500 ms without keepalive → all outputs to safe state
Sequence guard	Per-channel monotonic sequence rejects stale/duplicate packets
Run ID reset	New `run_id` resets all sequence counters to accept fresh commands
Timing guardrails	Period clamped to [100 ms, 30 s]; pulse clamped to [50 μs, $T/2$]
Servo hard cap	Frequencies > 2 Hz rejected; channel disabled
Peer re-registration	3 send failures → peer deleted and re-added
Duration auto-stop	Pressure channels auto-stop after configured duration
Event burst limit	Max 400 events per scheduler iteration (prevents infinite loops)

8.2 Safe Stop Behavior

Slave	Safe stop action
Electrical	All 12 DAC channels set to `neutral_code` (0x80 = 0 V)
Pressure	All 12 GPIO outputs set LOW (solenoids off)
Servo	All 12 servos driven to DEG_MIN (30°)

9 Memory and Performance

9.1 Web Server Heap Management

The Master's web UI avoids dynamic `String` allocation entirely. All HTML is generated using stack-allocated `char buf[]` arrays with `snprintf()` and streamed via `server.sendContent()`. The `Connection: close` header and explicit `server.client().stop()` ensure TCP sockets are released promptly.

The master dashboard displays real-time heap metrics (`ESP.getFreeHeap()` and `ESP.getMaxAllocHeap()`) for monitoring memory health.

9.2 Queue Sizing

Queue	Depth	Item size	Device
`msgQueue` (Electrical)	16	42 B	Electrical slave
`updateQueue`	36	∼28 B	Electrical slave
`msgQueue` (Pressure)	16	42 B	Pressure slave
`msgQueue` (Servo)	16	42 B	Servo slave

The queue depth of 16 accommodates the Master's burst of 12 `CMD_SET` messages (one per channel) plus headroom for keepalives arriving during processing. The Electrical slave's `updateQueue` has depth 36 (12×3) to hold 12 CONFIG + 12 START updates plus 12 spare.

10 Utility Sketches

10.1 BICEP_MAC_Address

A simple utility sketch that prints the ESP32's MAC address to Serial at 300 baud. Used during initial hardware provisioning to discover each board's MAC address for the Master's peer table.

```
void setup(){
  Serial.begin(300);
  Serial.print("ESP Board MAC Address: ");
  Serial.println(WiFi.macAddress());
}
```

10.2 Test_on_Wifi

A diagnostic sketch that creates a Wi-Fi AP and confirms the ESP32's radio is functional. Used for hardware bring-up testing.

11 Complete File Reference

File	Lines	Role
`Master_Pressure_Servo_Electrical.ino`	1049	Central controller: Wi-Fi AP, Web UI, ESP-NOW coordinator, state machine
`Electrical_Receiver_V2.ino`	755	Electrical slave: 12-ch DAC output, dual-task architecture, sub-ms timing
`Pressure_Receiver_V2.ino`	506	Pressure slave: 12-ch GPIO solenoid control, duration auto-stop
`Servo_Receiver_V2.ino`	494	Servo slave: 12-ch servo motor, ramp profile, 2 Hz hard cap
`BICEP_MAC_Address.ino`	16	Utility: print ESP32 MAC address
`Test_on_Wifi.ino`	—	Utility: Wi-Fi AP smoke test

12 Appendix: MAC Addresses (Current Configuration)

Device	MAC Address	Variable
Electrical slave	`94:54:C5:74:AB:38`	`mac_elec[]`
Pressure slave	`68:FE:71:0C:B9:50`	`mac_press[]`
Servo slave	`68:FE:71:0C:B7:F8`	`mac_motor[]`